Assignment 1

Kinematic Chains, Interpolation of Rotations and Camera Models

Virtual Humans — Winter Term 2026/27
University of Tübingen

October 2026

1 Introduction

Articulated body models of virtual humans are chains of rigid segments connected by joints; their pose changes over time, and they are observed through cameras. This assignment treats the elementary mathematics behind these three aspects: the forward kinematics of an articulated chain (Part 1), the interpolation of rotations between key orientations (Part 2), and the projection of three-dimensional points into an image by a pinhole camera (Part 3). Notation and conventions follow Lecture 02 (*Image Formation and Rotations*) [1]; the parts correspond to Lecture 02.2 (rotations and kinematic chains) and Lecture 02.1 (camera model).

Each part consists of *computational tasks*, in which you implement functions in Python, and *written questions*, in which you interpret the results and justify them mathematically. The points are distributed as shown in Table 1.

Table 1: Parts of the assignment and their points.

Part	Topic	Points	Lecture material
1	Planar kinematic chains	50	Lecture 02.2, slides 7–11, 42–43 and 46–50
2	Interpolation of rotations	25	Lecture 02.2, slides 12–19 and 28–32
3	Camera models	25	Lecture 02.1, slides 42–80
	Total	100	

1.1 Material

- `part_1.ipynb`, `part_2.ipynb`, `part_3.ipynb`: one Jupyter notebook per part, with the notation, the task descriptions, the code cells to complete and the cells for the written answers;
- `vh_a1.zip`: the helper package used by the notebooks (parameters of the scenes, plotting routines, given functions and the self-checks); it contains no solutions;
- `assignment_1_lab.html` (*optional*): interactive illustrations of the concepts of this assignment. It opens in any current web browser, works without an internet connection (only cosmetic web fonts are optional) and contains no solutions; nothing in the notebooks depends on it.

1.2 Working with the notebooks

The notebooks run in Google Colab or in a local Jupyter installation (Python 3 with NumPy and Matplotlib). Place `vh_a1.zip` next to the notebook; in Colab, the first cell asks you to upload it. Run the first cell of each notebook before any other cell.

- Code that you have to write is enclosed by `START OF YOUR SOLUTION` and `END OF YOUR SOLUTION`; replace each `None` placeholder inside these markers.
- A *check cell* follows each coding block. For every check it prints `passed`, `FAILED` (with the expected and the obtained value and a hint) or `not implemented`. The checks are a self-test against reference values; they do not determine the grade.
- Written questions are placed in cells headed **Answer**. Answer in complete sentences, and give a derivation where the question asks for one.
- Points and vectors are NumPy arrays, and a set of points is stored as rows (an $(n, 2)$ array for n planar points, an $(8, 3)$ array for the cube vertices, an $(n, 4)$ array for n quaternions). Names in the code follow the symbols of the text, for example `rot2` for $\mathbf{R}(\theta)$, `T` for $\mathbf{T}(\mathbf{q}, \theta)$ and `l1` for ℓ_1 .
- Use NumPy and Matplotlib only. Library routines for rotations, quaternions or camera projection (for example `scipy.spatial.transform`) must not replace the requested implementations; they follow conventions different from those fixed in Section 2.

1.3 Submission and grading

Work in teams of up to three students and submit one solution per team. Submit the three completed notebooks (`.ipynb`) together with a PDF export of each; the PDF must show all outputs, figures and written answers. The deadline and the submission procedure are announced on the course page. **Every submission must state the full names and the matriculation numbers of all team members at the very start of `part_1.ipynb`** (the cell headed *Team members*). Grading is based on the *correctness and clarity* of the code and of the written answers. The solution must be the team's own work; sources other than the lecture material have to be cited.

2 Notation and conventions

The following conventions hold throughout; each part adds the symbols it needs.

- Scalars are set in italics (a), vectors are bold lower-case column vectors ($\mathbf{a}$; the camera centre $\mathbf{C}$ keeps the capital of the slides), matrices are bold upper-case ($\mathbf{A}$). A bar over a point denotes homogeneous coordinates, $\bar{\mathbf{x}} = (\mathbf{x}^\top, 1)^\top$; over a quaternion it denotes the conjugate, $\bar{\mathbf{q}}$, as on the slides.
- Angles are in radians inside formulas and in the functions you implement; degrees (written 50°) occur in the text and in a few given data lists, and are converted by `np.deg2rad`.
- Rotations are *active* (they move points) and counter-clockwise in a right-handed coordinate system.

- S denotes the spatial (world) frame of Parts 1 and 2. In Part 3 the matrix $\mathbf{R}$ maps world coordinates to camera coordinates; it is the transpose of the matrix whose columns are the camera axes.
- Some symbols are used with a part-local meaning, as on the slides: $\mathbf{q}$ is a joint position in Part 1 and a quaternion in Part 2; every part introduces its symbols before they are used.

3 Part 1: Planar kinematic chains (50 points)

3.1 Rotation about a point and the kinematic chain

In a kinematic chain, rotating one joint moves everything attached to it. A point $\mathbf{x}$ in the plane that is rotated by the angle θ about the centre $\mathbf{q}$ of a revolute joint moves to

$$\mathbf{x}' = \mathbf{q} + \mathbf{R}(\theta)\,(\mathbf{x} - \mathbf{q}), \qquad \mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}. \tag{1}$$

In homogeneous coordinates this is a single matrix product, $\bar{\mathbf{x}}' = \mathbf{T}(\mathbf{q}, \theta)\,\bar{\mathbf{x}}$, with

$$\mathbf{T}(\mathbf{q}, \theta) = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{q} - \mathbf{R}(\theta)\mathbf{q} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \exp(\hat{\boldsymbol{\xi}}\theta), \tag{2}$$

the exponential of the twist $\hat{\boldsymbol{\xi}}$ of a revolute joint located at $\mathbf{q}$ (Lecture 02.2, slides 42–43; the chain below: slides 49–50).

Consider a chain of k revolute joints. Let $\mathbf{q}_i$ be the position of joint i at rest (all joint angles zero) and θ_i the angle of joint i *relative to its parent*, so that everything beyond joint i turns by θ_i about $\mathbf{q}_i$. A point $\mathbf{p}$ on the link beyond joint k, located at $\bar{\mathbf{p}}(\mathbf{0})$ at rest, is then at

$$\bar{\mathbf{p}}(\theta_1, \ldots, \theta_k) = \mathbf{T}_1(\theta_1)\,\mathbf{T}_2(\theta_2)\cdots\mathbf{T}_k(\theta_k)\,\bar{\mathbf{p}}(\mathbf{0}), \qquad \mathbf{T}_i(\theta_i) = \mathbf{T}(\mathbf{q}_i, \theta_i). \tag{3}$$

This is the product of exponentials formula [2]. All $\mathbf{T}_i$ are built from the *rest* positions $\mathbf{q}_i$. Read from the right, joint k first turns its link about $\mathbf{q}_k$, then joint $k-1$ turns that link together with everything beyond it, and so on.

3.2 The robot

The arm starts at the rest position $\mathbf{q}_1 = (-0.4,\ 0.3)^\top$ of joint 1. Its rigid links are given by the rest link vectors $\mathbf{a}_i = \mathbf{q}_{i+1} - \mathbf{q}_i$; the last link ends in the arm end $\mathbf{e}$, to which a claw is rigidly attached. The claw tips are $\mathbf{e} + \mathbf{c}^+$ and $\mathbf{e} + \mathbf{c}^-$ with $\mathbf{c}^\pm = (0.15,\ \pm 0.15)^\top$. The arms used in the tasks are:

- one joint: $\mathbf{a} = (0.7,\ 0)^\top$ (Tasks 1.1 and 1.2);
- two joints: $\mathbf{a}_1 = (0.7,\ 0)^\top$ and $\mathbf{a}_2 = (0.25,\ 0)^\top$ (Tasks 1.3 and 1.4);
- three joints: three links $\mathbf{a}_i = (0.35,\ 0)^\top$ (Task 1.5).

All positions refer to the spatial frame S. The workspace plots of Task 1.4 show the window $[-3, 3]^2$; the other plots are zoomed in. The rest configurations are shown in Figure 1.

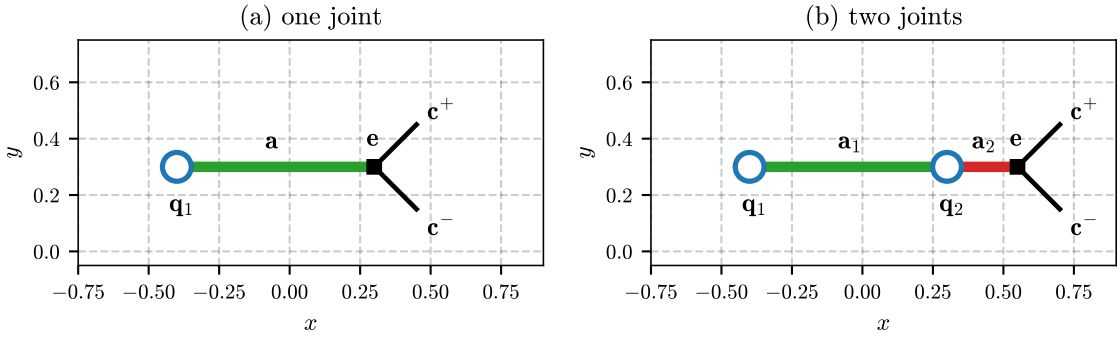


Figure 1: Rest configuration of the arm (all joint angles zero): (a) one joint, (b) two joints. Joint positions $\mathbf{q}_i$ (circles), links $\mathbf{a}_i$, arm end $\mathbf{e}$ (square) and claw tips $\mathbf{e} + \mathbf{c}^{\pm}$.

3.3 Tasks

Task 1.1 (5 points): rotation about a joint. Implement $\mathbf{R}(\theta)$ and Equation (1). Rotate the arm end $\mathbf{e}$ and both claw tips about $\mathbf{q} = \mathbf{q}_1$ by $\theta = 50°$.
Questions. Which representation of the rotation did you use, and on what does it act? Why does $\mathbf{q}$ not change? How are $\mathbf{e}$ and the claw tips updated, and why? (Show that the tip $\mathbf{e} + \mathbf{c}^{\pm}$ moves to $\mathbf{e}' + \mathbf{R}(\theta)\mathbf{c}^{\pm}$.) Why is $\mathbf{q}$ subtracted in (1) instead of computing $\mathbf{R}(\theta)\mathbf{x}$?

Task 1.2 (5 points): reachable positions of one joint. Sample $N = 50$ equally spaced angles $\alpha_j = 2\pi j/N$, $j = 0, \dots, N-1$ (without repeating 2π) and collect the arm-end positions $\mathbf{e}(\alpha_j)$.
Questions. Give $\mathbf{e}(\alpha)$ in closed form in terms of $\mathbf{q}$ and $\mathbf{a}$. Which set of points does it trace?

Task 1.3 (12 points): two joints. Implement $\mathbf{T}(\mathbf{q}, \theta)$ as a 3×3 matrix, Equation (2). Compute the new positions of joint 2, the arm end and the two claw tips for $(\theta_1, \theta_2) = (70°, 0°)$ and for $(70°, 40°)$. For $(70°, 40°)$ also compare the two products $\mathbf{T}_1(\theta_1)\mathbf{T}_2(\theta_2)$ and $\mathbf{T}_2(\theta_2)\mathbf{T}_1(\theta_1)$.
Questions. Which of $\mathbf{T}_1, \mathbf{T}_2$ act on $\mathbf{q}_1$, on $\mathbf{q}_2$, and on $\mathbf{e}$ and the claw tips? Why does $\mathbf{T}_2$ not move $\mathbf{q}_2$ whereas $\mathbf{T}_1$ does? Why is $\mathbf{T}_1\mathbf{T}_2 \neq \mathbf{T}_2\mathbf{T}_1$ although planar rotations $\mathbf{R}(\theta_1)\mathbf{R}(\theta_2) = \mathbf{R}(\theta_2)\mathbf{R}(\theta_1)$ commute, and which order is the correct chain? Is the arm end the same if the two angles are swapped, $(\theta_1, \theta_2) = (40°, 70°)$? Argue with $\mathbf{e} = \mathbf{q}_1 + \mathbf{R}(\theta_1)\mathbf{a}_1 + \mathbf{R}(\theta_1 + \theta_2)\mathbf{a}_2$.

Task 1.4 (16 points): workspace of two joints. Let both joints sweep a full turn on the grid $\vartheta_j = 2\pi j/N$, $j = 0, \dots, N-1$, $N = 100$, for each joint ($N^2 = 10\,000$ configurations), and collect the arm end for every pair (θ_1, θ_2) of grid values. Write this computation as a function of the link vectors. Then (i) make the second link longer than the first, $\mathbf{a}_2 = (\ell_2, 0)^\top$ with $\ell_2 > \ell_1 = 0.7$, and repeat; (ii) find link lengths ℓ_1, ℓ_2 for which the arm reaches essentially the whole window $[-3, 3]^2$. The notebook measures the fraction of the window (in cells of side 0.2) that contains at least one of your points; the self-check asks for at least 99.5 %.
Questions. What does the reachable set look like? Give it in closed form in terms of $\ell_1 = \|\mathbf{a}_1\|$, $\ell_2 = \|\mathbf{a}_2\|$ and $\mathbf{q}_1$; which points are not reachable? How does the set change for a longer second link, and what happens at $\ell_2 = \ell_1$ and beyond? Which lengths did you choose to cover the window, and why (derive them)?

Task 1.5 (12 points): three joints. Write *one* function that moves any rest point $\mathbf{p}$ lying beyond the first k joints according to Equation (3), and use it for the three-joint arm with $(\theta_1, \theta_2, \theta_3) = (40°, -80°, 40°)$. Record two further angle triples (in degrees) that reach the same

arm end $\mathbf{e}$: a *different* triple with the same arm end, and a different triple with the same arm end *and* the same claw orientation.

Questions. Which angles did you choose and how did you find them? How many configurations reach the same arm end (count degrees of freedom and constraints)? How many if the claw orientation must also agree, and why?

4 Part 2: Interpolation of rotations (25 points)

4.1 Euler angles and quaternions

The *Euler angles* (α, β, γ) of this assignment define the rotation

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha)\, \mathbf{R}_y(\beta)\, \mathbf{R}_z(\gamma),$$

where, in active form (Lecture 02.2, slides 14 and 17),

$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}, \quad \mathbf{R}_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}, \quad \mathbf{R}_z(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

This is the composition of intrinsic rotations about x, then y', then z''. For $\beta = \pm\pi/2$ the axes of α and γ coincide (gimbal lock): $\mathbf{R}$ then depends only on $\alpha + \gamma$ (at $+\pi/2$) or on $\alpha - \gamma$ (at $-\pi/2$), and one degree of freedom is lost.

A *quaternion* $\mathbf{q} = (q_w, \mathbf{v})$ with $\mathbf{v} = (q_x, q_y, q_z)^\top$ is written scalar part first. The rotation by the angle θ about the unit axis $\boldsymbol{\omega}$ is represented by the unit quaternion $\mathbf{q} = (\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\boldsymbol{\omega})$; the quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation. The product of two quaternions $\mathbf{p} = (p_w, \mathbf{v}_p)$ and $\mathbf{q} = (q_w, \mathbf{v}_q)$ is (slide 30, where it is written $\mathbf{q}_1 \circ \mathbf{q}_2$)

$$\mathbf{p} \circ \mathbf{q} = \big(p_w q_w - \mathbf{v}_p \cdot \mathbf{v}_q,\ \ p_w \mathbf{v}_q + q_w \mathbf{v}_p + \mathbf{v}_p \times \mathbf{v}_q\big),$$

the conjugate is $\bar{\mathbf{q}} = (q_w, -\mathbf{v})$ and the inverse is $\mathbf{q}^{-1} = \bar{\mathbf{q}} / \|\mathbf{q}\|^2$. The logarithm and the exponential are

$$\ln \mathbf{q} = \Big(\ln\|\mathbf{q}\|,\ \tfrac{\mathbf{v}}{\|\mathbf{v}\|} \arccos\tfrac{q_w}{\|\mathbf{q}\|}\Big), \qquad \exp(q_w, \mathbf{v}) = e^{q_w}\Big(\cos\|\mathbf{v}\|,\ \tfrac{\mathbf{v}}{\|\mathbf{v}\|}\sin\|\mathbf{v}\|\Big),$$

and real powers are defined through them, $\mathbf{q}^t = \exp(t \ln \mathbf{q})$. The given function `quat_to_matrix` applies the sandwich product $\mathbf{q} \circ (0, \mathbf{x}) \circ \bar{\mathbf{q}}$ (with the conjugate, not the inverse) *without* normalising $\mathbf{q}$; a quaternion that is not of unit length therefore scales the rotated object by $\|\mathbf{q}\|^2$.

4.2 Interpolation methods

Between a start and an end orientation, n frames are generated at the parameters $t_k = k/(n-1)$, $k = 0, \dots, n-1$ (both end points are included). The simplest scheme is linear interpolation,

$$\text{value}(t_k) = (1 - t_k)\,\text{start} + t_k\,\text{end}. \tag{1}$$

Euler-lerp applies it to the three Euler angles $\boldsymbol{\eta} = (\alpha, \beta, \gamma)^\top$ and converts the result to a rotation,

$$\boldsymbol{\eta}(t_k) = (1 - t_k)\,\boldsymbol{\eta}_0 + t_k\,\boldsymbol{\eta}_1, \qquad \mathbf{R}_k = \mathbf{R}\big(\boldsymbol{\eta}(t_k)\big). \tag{2}$$

Lerp applies it to the four components of the quaternions,

$$\mathbf{q}(t_k) = (1 - t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1. \tag{3}$$

Nlerp (normalised Lerp) projects the result back onto the unit sphere,

$$\mathbf{q}(t_k) = \frac{(1 - t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1}{\left\lVert (1 - t_k)\,\mathbf{q}_0 + t_k\,\mathbf{q}_1 \right\rVert}. \tag{4}$$

Slerp (spherical linear interpolation, for unit quaternions) follows a great circle of the unit sphere with constant angular velocity,

$$\mathbf{q}(t) = \mathbf{q}_0 \circ \left(\mathbf{q}_0^{-1} \circ \mathbf{q}_1\right)^{t}, \qquad t \in [0, 1]. \tag{5}$$

Slerp is due to Shoemake [3].

4.3 Test object and test sequences

All interpolations are shown on the letter F. A *sequence* is a list of waypoints given as Euler angles in degrees; between neighbouring waypoints an interpolation produces $n = 40$ frames, so every segment has the same number of frames. Table 2 lists the four test sequences, and Figure 2 shows the F at their waypoints.

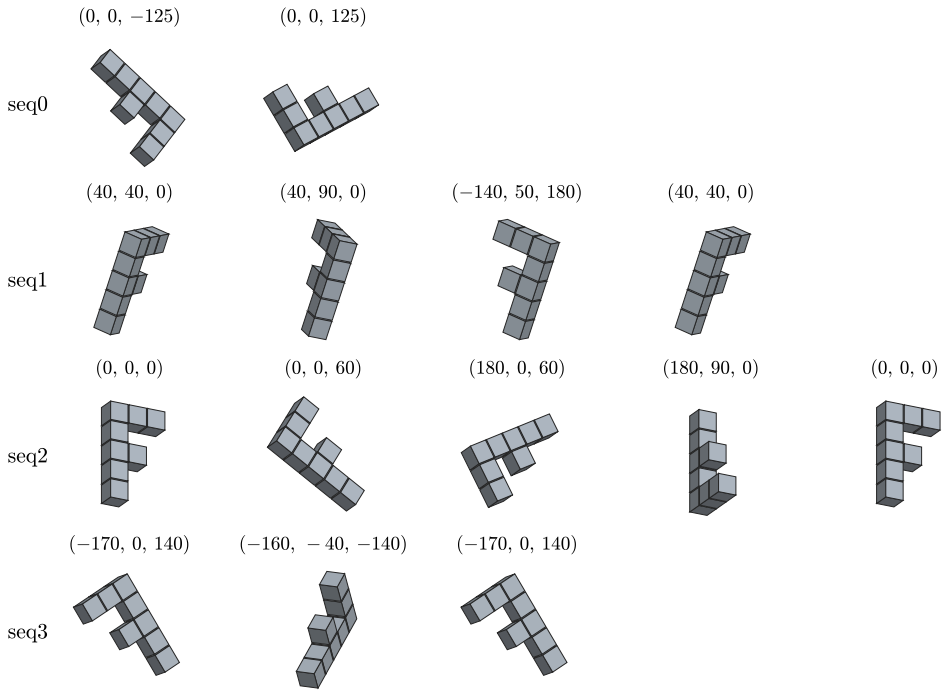


Figure 2: The F-object at the waypoints of the test sequences; the titles give (α, β, γ) in degrees.

Table 2: Test sequences; waypoints are Euler angles (α, β, γ) in degrees.

Name	Waypoints	Property
`seq0`	$(0,\ 0,\ -125),\ (0,\ 0,\ 125)$	one axis only: γ from -125° to 125°
`seq1`	$(40,\ 40,\ 0),\ (40,\ 90,\ 0),\ (-140,\ 50,\ 180),\ (40,\ 40,\ 0)$	several angles change at once; the path passes through $\beta = 90^\circ$
`seq2`	$(0,\ 0,\ 0),\ (0,\ 0,\ 60),\ (180,\ 0,\ 60),\ (180,\ 90,\ 0),\ (0,\ 0,\ 0)$	contains two segments in which the object turns by exactly 180°
`seq3`	$(-170,\ 0,\ 140),\ (-160,\ -40,\ -140),\ (-170,\ 0,\ 140)$	two orientations that are only about 85° apart, but written with γ values that are 280° apart

4.4 Tasks

Task 2.1 (5 points): Euler-lerp. Implement the linear interpolation of Euler angles, Equation (2); the conversion to rotation matrices is given.
Questions. Describe `seq0` and `seq1`: is the path the shortest one, does the object turn without jerks within a segment, is the angular speed constant within a segment? What is wrong with interpolating the three angles independently?

Task 2.2 (5 points): quaternion interpolation. Implement Lerp, Equation (3), for quaternions in scalar-first order. Do *not* normalise.
Questions. Compare with Task 2.1 on `seq1`. Which artefact appears on `seq2`, and where is it strongest? Explain it: what is $\|\mathbf{q}(t)\|$ along a segment, and what does `quat_to_matrix` do with it?

Task 2.3 (5 points): Nlerp. Implement Equation (4).
Questions. What does Nlerp fix compared with Task 2.2, and which artefact remains visible in the speed plot? Is the angular speed constant, and where is the F fastest?

Task 2.4 (10 points): Slerp. (a) *Quaternion algebra (4 points).* Implement the product, the logarithm and the exponential of Section 4.1; the degenerate case $\mathbf{v} = \mathbf{0}$ is handled for you. (b) *Slerp (3 points).* Implement Equation (5) exactly as written. (c) *Shortest path (3 points).* Slerp follows a great circle, but is the result also the shortest rotation between the two orientations? Measure the swept angle on `seq0` and `seq3`, and implement `slerp_shortest`, a Slerp that always takes the shorter of the two possible paths.
Questions. What does Slerp do better than Nlerp? Compare the speed curves on `seq1` and on the 180° segments of `seq2`; is the speed constant? Why do $\mathbf{q}$ and $-\mathbf{q}$ give the same rotation? Towards which of the two does Equation (5) interpolate, and when is that the longer way? How large is the detour (angle swept as written minus shortest angle) for the pair of `seq0` and for the first pair of `seq3`? State the rule you implemented in `slerp_shortest` and explain why it works.

5 Part 3: Camera models (25 points)

5.1 The pinhole camera

The notation is that of Lecture 02.1. A point has world coordinates $\mathbf{p}_W = (U, V, W)^\top$ and camera coordinates $\mathbf{p}_C = (X, Y, Z)^\top$ (written $\mathbf{P}_W$, $\mathbf{P}_C$ on the slides), where X points to the right, Y down and Z along the optical axis. The camera has its centre at $\mathbf{C}$ (in world coordinates), and the rows of the rotation matrix $\mathbf{R}$ are the camera axes expressed in world coordinates (slides 66–68):

$$\mathbf{p}_C = \mathbf{R}\,(\mathbf{p}_W - \mathbf{C}).$$

With the focal length f (a length, in the units of the scene), the pixel sizes s_x, s_y (the effective scale factors of slide 76) and the principal point (o_x, o_y), the film coordinates are $(x, y) = \big(fX/Z,\ fY/Z\big)$ and the pixel coordinates (u, v) of the image point — the pixel axis v points *down* — are

$$u = \frac{f}{s_x}\,\frac{X}{Z} + o_x, \qquad v = \frac{f}{s_y}\,\frac{Y}{Z} + o_y,$$

or, with the intrinsic matrix

$$\mathbf{K} = \begin{bmatrix} f/s_x & 0 & o_x \\ 0 & f/s_y & o_y \\ 0 & 0 & 1 \end{bmatrix},$$

in homogeneous form, with $\lambda = Z$. The complete projection of a world point is therefore

$$\mathbf{p}_C = \mathbf{R}\,(\mathbf{p}_W - \mathbf{C}), \qquad \begin{bmatrix} \lambda u \\ \lambda v \\ \lambda \end{bmatrix} = \mathbf{K}\,\mathbf{p}_C, \qquad (u, v) = \Big(\tfrac{\lambda u}{\lambda},\ \tfrac{\lambda v}{\lambda}\Big). \tag{1}$$

(Lecture 02.1, slides 51–58 and 76–77.) The pinhole model is treated in detail in [4].

5.2 The scene

The image has 640×480 pixels. The camera has the focal length $f = 1$ and pixel size $s_x = s_y = 0.0025$, so that $f/s_x = 400$ px; its principal point is $(o_x, o_y) = (220,\ 160)$ px, deliberately *not* the image centre. The camera centre is $\mathbf{C} = (2,\ -6,\ 1.5)^\top$ and the camera looks along $+V$ with W pointing up,

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Two rectangles and a cube are given. `rect1` lies in the plane $V = 3$, parallel to the image plane, with corners $(-1,\ 3,\ 1.5)$, $(-1,\ 3,\ 3)$, $(1,\ 3,\ 3)$, $(1,\ 3,\ 1.5)$. `rect2` lies on the ground $W = 0$, perpendicular to the image plane, with corners $(-1,\ 0,\ 0)$, $(3,\ 0,\ 0)$, $(3,\ 10,\ 0)$, $(-1,\ 10,\ 0)$. The cube has the edge length $L = 3$; its vertices $\mathbf{x}_1, \ldots, \mathbf{x}_8$ are handed out in a tilted initial pose, where $\mathbf{x}_1, \ldots, \mathbf{x}_4$ form the *front face* (the face nearer to the camera) and $\mathbf{x}_{i+4}$ lies behind $\mathbf{x}_i$. Figure 3 shows the scene and its image.

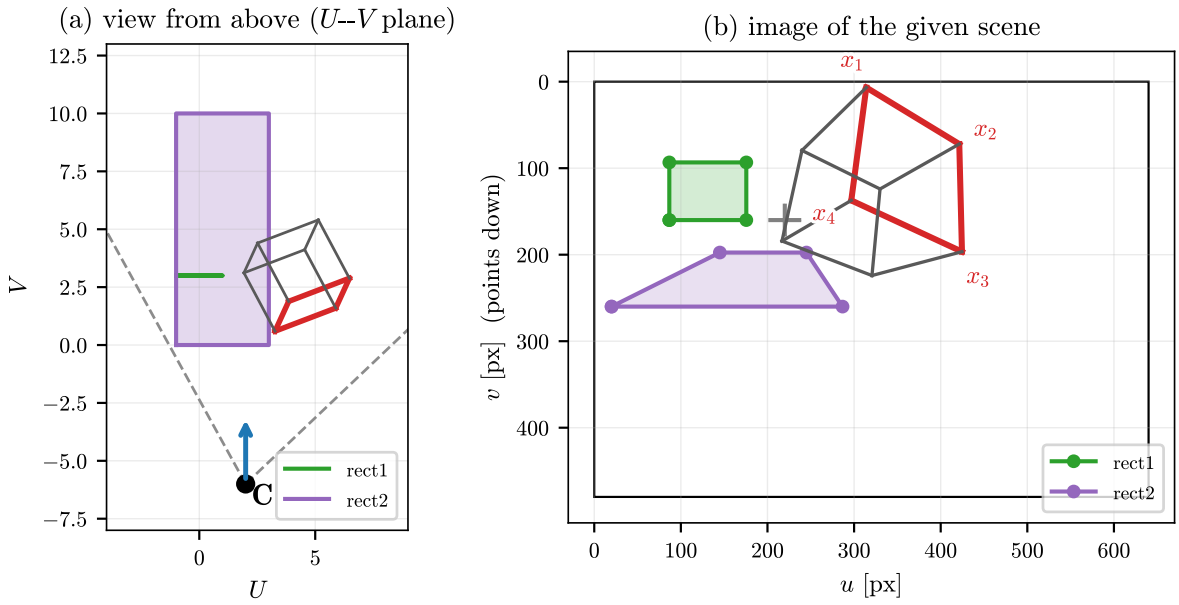


Figure 3: The given scene. (a) View from above: camera centre $\mathbf{C}$, optical axis (arrow), horizontal field of view (dashed), the footprints of `rect1` and `rect2` and the cube. (b) Their image; the front face of the cube is drawn in red, the cross marks the principal point (o_x, o_y). Image axes: u to the right, v down.

5.3 Tasks

Task 3.1 (5 points): projection and camera motion. (a) *(2 points)* Implement `project(K, R, C, p_W)`, which returns the pixel (u, v) of a world point according to Equation (1); assume the point is in front of the camera, $Z > 0$. (b) *(1 point)* Define `rect3`, a rectangle of size 3×2 in a plane $U = \text{const}$ (parallel to the V–W plane, like a wall), 3 long along V and 2 along W, given by its four corners (in order around the rectangle) and completely visible in the image. (c) *(2 points)* Rotate the camera *body* by 20° about its own X axis, $\mathbf{Q} = \mathbf{R}_x(20^\circ)$ (the camera axes, i.e. the rows of $\mathbf{R}$, turn with it, so that the new extrinsic rotation is $\mathbf{R}' = \mathbf{Q}^\top \mathbf{R}$), and store $\mathbf{R}'$ in `R_tilt`; translate the camera by $+3$ units along the world U axis and store the new camera centre in `C_shift` (everything else stays the same).

Questions. Compare the images of the three rectangles: which stays a rectangle, which become trapezoids, which edges remain parallel, and why? (Think of lines along V, the viewing direction.) What happens to the image when the camera is turned about its X axis, and what when it is translated along U?

Task 3.2 (15 points): pose of a 3D object. Camera and intrinsics stay fixed. Find a *rigid* motion of the cube, $\mathbf{x} \mapsto \mathbf{R}_{\text{obj}}\mathbf{x} + \mathbf{t}_{\text{obj}}$ (no scaling, no mirroring), such that the front face is projected as an axis-aligned square of side 120 px (which vertex lands on which corner of the square does not matter), and the cube stays in front of the camera and inside the image with the front face nearer than the back face. Implement `cube_pose(V, K, R, C, side_px)`, which returns $(\mathbf{R}_{\text{obj}}, \mathbf{t}_{\text{obj}})$.

Hints. Build an orthonormal frame from the edges of the cube and align it with the camera axes. By similar triangles, the side of the face in pixels is $\frac{f}{s_x}\frac{L}{Z_0}$; solve for the depth Z_0 and place the face at that depth.

Questions. Describe your construction in a few sentences: which frame did you align with which,

and how did you obtain the translation? How many solutions are there? Which freedoms of the pose does the requirement fix, and which remain free (count the discrete choices, describe the continuous ones)? If the centre of the front face must in addition lie on the optical axis, how many solutions remain?

Task 3.3 (5 points): camera intrinsics. Keep the cube pose of Task 3.2 and the camera pose $(\mathbf{R}, \mathbf{C})$, and change *only* the intrinsics f (a length; $f = 1$ at the start) and (o_x, o_y); the pixel size stays $s_x = s_y$. Choose them such that the front face is again an axis-aligned square which is (i) doubled (side 240 px) and centred in the image (centre (320, 240), the middle of the image), (ii) halved (side 60 px) and centred in the image, and (iii) doubled and centred for the *moved* cube `V_shift`, which is the cube of Task 3.2 shifted by 0.5 units along the world U axis (given in the notebook). Do not assume that (o_x, o_y) is the image centre: it depends on where the square lands.
Questions. Which f and (o_x, o_y) did you choose in each case, and why (derive them)? Why is it wrong in general to take the image centre as (o_x, o_y); give the formula for (o_x, o_y) in terms of the camera coordinates $(X_c, Y_c, Z_c)^\top$ of the centre of the square. What changes in the picture if you double f with the camera fixed, and what change of the camera pose gives (for the front face of the cube) the same size?

6 Checklist before submission

- All three notebooks run from top to bottom without errors (restart the runtime and run all cells).
- Every `None` placeholder in the marked regions has been replaced; all outputs and figures are visible.
- Every **Answer** cell contains a written answer; derivations are shown where requested.
- For each notebook, a PDF export showing all outputs, figures and answers is included.
- The names and matriculation numbers of all team members (at most three) are entered at the start of `part_1.ipynb`.

References

[1] Lecture slides of *Virtual Humans*, Lecture 02.1 (*Image Formation*; focal length versus viewpoint: slides 36–39) and Lecture 02.2 (*Rotations and Kinematic Chains*), Winter Term 2026/27.

[2] R. M. Murray, Z. Li and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation.* CRC Press, 1994.

[3] K. Shoemake. Animating rotation with quaternion curves. *Computer Graphics (Proc. SIGGRAPH)*, 19(3):245–254, 1985.

[4] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision.* Cambridge University Press, 2nd edition, 2004.